62. Вывод типов: ключевое слово auto

На этом уроке мы рассмотрим вывод типов с помощью ключевого слова auto в языке C++.

Оглавление:

1. Вывод типов в C++11
2. Ключевое слово auto и параметры функций
3. Вывод типов в C++14
4. trailing-синтаксис в C++11
5. Заключение

Вывод типов в C++11

До C++11 ключевое слово auto было наименее используемым ключевым словом в языке C++. Мы уже знаем, что локальные переменные имеют автоматическую продолжительность жизни (создаются в точке определения и уничтожаются в конце блока, в котором определены).

Ключевое слово auto использовалось для явного указания, что переменная должна иметь автоматическую продолжительность жизни:

```
int main()
{
    auto int boo(7); // явно указываем, что переменная boo типа int должна иметь автоматическую продолжительность жизни

    return 0;
}
```

Однако, поскольку все переменные в новых версиях языка C++ по умолчанию имеют автоматическую продолжительность жизни (если явно не указать другой **тип продолжительности жизни**), ключевое слово auto стало лишним и, следовательно, устаревшим.

В C++11 значение ключевого слова auto изменилось. Рассмотрим следующий стейтмент:

```
double x = 4.0;
```

Если C++ и так знает, что `4.0` является литералом **типа double**, то зачем нам дополнительно указывать, что переменная `x` должна быть типа double? Правда, было бы неплохо, если бы мы могли указать переменной принять соответствующий тип данных, основываясь на инициализируемом значении?

Начиная с C++11 **ключевое слово auto** при инициализации переменной может использоваться вместо типа переменной, чтобы сообщить компилятору, что он должен присвоить тип переменной исходя из инициализируемого значения. Это называется **выводом типа** (или ***«автоматическим определением типа данных компилятором»***). Например:

```
auto x = 4.0; // 4.0 - это литерал типа double, поэтому и переменная x должна быть типа double
auto y = 3 + 4; // выражение 3 + 4 обрабатывается как целочисленное, поэтому и переменная y должна быть типа int
```

Это работает даже с возвращаемыми значениями функций:

```
int subtract(int a, int b)
{
    return a - b;
}

int main()
{
    auto result = subtract(4, 3); // функция subtract() возвращает значение типа int и, следовательно, переменная result также
    return 0;
}
```

Обратите внимание, это работает только с инициализированными переменными. Переменные, объявленные без инициализации, не могут использовать эту особенность (поскольку нет инициализируемого значения, и компилятор не может знать, какой тип данных присвоить переменной).

Используя ключевое слово auto вместо фундаментальных типов данных, мы не сэкономим много времени или усилий, но на следующих уроках, когда типы данных будут более сложными и длинными, ключевое слово auto может очень пригодиться.

Ключевое слово auto и параметры функций

Многие новички пытаются сделать что-то вроде следующего:

```
void mySwap(auto a, auto b)
{
    auto x = a;
    a = b;
    b = x;
}
```

Это не сработает, так как компилятор не может определить типы данных для параметров функции `a` и `b` во время компиляции.

Если вы хотите создать функцию, которая будет работать с разными типами данных, то вам лучше воспользоваться шаблонами функций, а не выводом типа. Это ограничение, возможно, отменят в будущих версиях C++ (когда auto будет использоваться как сокращенный способ создания шаблонов функций), но в C++14 это не работает. Единственное исключение — **лямбда-выражения** (но это уже другая тема).

Вывод типов в C++14

В C++14 функционал ключевого слова auto был расширен до автоматического определения типа возвращаемого значения функции. Например:

```
auto subtract(int a, int b)
{
    return a - b;
}
```

Так как выражение `a - b` является типа int, то компилятор делает вывод, что и функция должна быть типа int.

Хотя это может показаться удобным, так делать не рекомендуется. Тип возвращаемого значения функции помогает понять caller-у, что именно функция должна возвращать. Если конкретный тип не указан, то caller может неверно интерпретировать тип возвращаемого значения, что может привести к непреднамеренным ошибкам.

Так почему же использование auto при инициализации переменных — это хорошо, а с функциями — плохо? Дело в том, что auto можно использовать при определении переменной, так как значение, из которого компилятор делает выводы о типе переменной, находится прямо там — в правой части стейтмента. Однако с функциями это не так — нет контекста, который бы указывал, какого типа данных будет возвращаемое значение. Фактически, пользователю придется лезть в тело функции, чтобы определить тип возвращаемого значения. Следовательно, такой способ не только не практичен, но и более подвержен ошибкам.

trailing-синтаксис в C++11

В C++11 появилась возможность использовать **синтаксис типа возвращаемого значения trailing** (или просто ***«trailing-синтаксис»***), когда компилятор делает выводы о типе возвращаемого значения по конечной части **прототипа функции**. Например, рассмотрим следующее объявление функции:

```
int subtract(int a, int b);
```

В C++11 это можно записать как:

```
auto subtract(int a, int b) -> int;
```

В этом случае auto не выполняет вывод типа — это всего лишь часть синтаксиса типа возвращаемого значения trailing. Зачем это стоит использовать? В основном, из-за удобства. Например, можно выстроить в колонку все названия ваших функций:

```
auto subtract(int a, int b) -> int;
auto divide(double a, double b) -> double;
auto printThis() -> void;
auto calculateResult(int a, double x) -> std::string;
```

Но этот синтаксис более полезен в сочетании с некоторыми другими продвинутыми особенностями языка C++ такими, как классы и ключевое слово decltype.

На данный момент я рекомендую придерживаться традиционного (обычного) синтаксиса для определения типа возвращаемого значения функции (без использования ключевого слова auto).

Заключение

Начиная с C++11 ключевое слово auto может использоваться вместо типа переменной при инициализации для выполнения вывода типа. Во всех других случаях использования ключевого слова auto следует избегать, если на это нет веских причин.